

Apresentação da Disciplina e Revisão de Complexidade

COMP0498 — Algoritmos e Estruturas de Dados II · Turma T02 · 2026.2

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP — Universidade Federal de Sergipe

18 de agosto de 2026

Parte I — A Disciplina (~40 min)

- O que vamos estudar e por quê
- As três unidades e o cronograma
- Avaliação, metodologia e bibliografia

Parte II — Revisão de Complexidade (~50 min)

- Por que analisar o pior caso
- Algoritmos iterativos: regras de contagem
- Algoritmos recursivos: recorrências
- Teorema Mestre (visão rápida)

Parte I — A Disciplina

Disciplina	COMP0498 — Algoritmos e Estruturas de Dados II
Carga horária	60h (30h teóricas + 30h práticas) · 4 créditos
Pré-requisito	COMP0497 — Algoritmos e Estruturas de Dados I
Horário	Terças e quintas-feiras, 35T34
Período letivo	18/08/2026 a 17/12/2026
Professor	André Yoshiaki Kashiwabara
Contato	andreyoshiaki@dcomp.ufs.br

Objetivo geral

Capacitar você a **projetar** e **analisar** algoritmos empregando as principais técnicas de projeto — força bruta, divisão e conquista, programação dinâmica, método guloso, backtracking e branch and bound —, com ênfase em algoritmos sobre **grafos**, na demonstração de **corretude**, na análise de **complexidade** e nas noções de **NP-completude**.

Em AED1 você aprendeu a *usar* estruturas de dados e algoritmos prontos. Aqui, o foco muda: dado um problema novo, **qual técnica escolher, como provar que a solução funciona e quanto ela custa?**

Unidade 1

Corretude: invariantes de laço
Força bruta e busca exaustiva
Quickselect e geração combinatória
Grafos: representação, DFS, BFS, ordenação topológica

1ª Avaliação: **29/09**

Unidade 2

Recorrências e método mestre
Divisão e conquista: Karatsuba e Strassen
Programação dinâmica: mochila, LCS, distância de edição, Warshall e Floyd

2ª Avaliação: **29/10**

Unidade 3

Guloso: Huffman, Kruskal, Prim, Dijkstra
Fluxo máximo e emparelhamento
NP-completude: P, NP, reduções
Backtracking e branch and bound

3ª Avaliação: **08/12**

Data	Conteúdo
18/08	Apresentação e revisão de pré-requisitos (hoje)
20/08 e 25/08	Corretude: invariantes de laço e terminação
27/08	Força bruta e busca exaustiva
01/09	Seleção e estatística de ordem (quickselect, mediana)
03/09	Geração combinatória: subconjuntos e permutações
08/09	<i>Sem aula — feriado municipal (São Cristóvão)</i>
10/09	Grafos: conceitos e representações
15/09 e 17/09	Busca em profundidade (DFS)
22/09	Busca em largura (BFS) e ordenação topológica
24/09	Exercícios e revisão
29/09	1ª Avaliação

Data	Conteúdo
01/10	Recorrências e método mestre: diagnóstico e limites
06/10	Multiplicação de inteiros grandes (Karatsuba)
08/10	Multiplicação de matrizes (Strassen)
13/10	Programação dinâmica: memoization e subestrutura ótima
15/10	Problema da mochila
20/10	Subsequência comum máxima (LCS) e distância de edição
22/10	PD em grafos: algoritmos de Warshall e de Floyd
27/10	Exercícios e revisão
29/10	2ª Avaliação

Data	Conteúdo
03/11	Método guloso: códigos de Huffman e argumento de troca
05/11	Árvore geradora mínima: Kruskal e Prim
10/11	Caminhos mínimos: Dijkstra
12/11 e 17/11	Fluxo máximo (Ford-Fulkerson) e emparelhamento bipartido
19/11 a 26/11	NP-completude: classes P/NP, reduções, exercícios
01/12	Backtracking e branch and bound
03/12	Exercícios e revisão; lista de reduções polinomiais
08/12	3ª Avaliação
15/12	Avaliação de reposição; revisão final
17/12	Encerramento e consolidação das notas

Três provas escritas individuais, uma por unidade

$$MF = \frac{N_1 + N_2 + N_3}{3}$$

- **N1** (29/09) e **N2** (29/10): prova escrita individual.
- **N3** (08/12): prova escrita individual (**70%**) + lista de reduções polinomiais desenvolvida em casa (**30%**).
- **Reposição** (15/12): para quem faltou a uma avaliação por motivo justificado; a nota substitui a da avaliação perdida.

Aprovação

Média final $\geq 5,0$ e frequência mínima de 75% da carga horária.

Em sala

- Aulas presenciais, preferencialmente em laboratório.
- Teoria com slides e quadro.
- Implementações em **Linguagem C**, na IDE de sua preferência.
- Exercícios resolvidos em sala.

Fora de sala

- Exercícios e estudos dirigidos como trabalho de casa.
- **SIGAA**: envio de atividades e trabalhos.
- **Classroom**: material de cada aula disponibilizado após a aula.

Básica:

- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 3ª ed., MIT Press, 2009.
- Sedgewick, Wayne — *Algorithms*, 4ª ed., Addison-Wesley, 2011.
- Levitin — *Introduction to the Design and Analysis of Algorithms*, 3ª ed., Pearson, 2012.

O CLRS é a referência principal para a maior parte do curso.

Estruturas de dados

- Vetores, listas, pilhas e filas
- Árvores de busca
- Fila de prioridade (heap)
- Union-find

Análise de algoritmos

- Notação assintótica (O , Ω , Θ)
- Análise de laços (iterativos)
- Recorrências (recursivos)
- Ordenação: Merge Sort, Quick Sort

Agora

Vamos revisar a coluna da direita — **análise de complexidade** —, que será usada em *todas* as aulas deste curso.

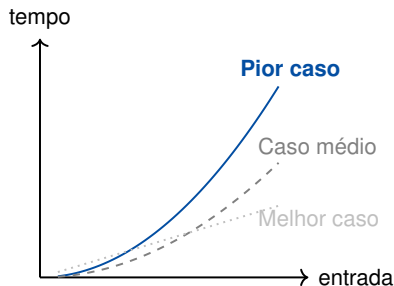
Parte II — Por que Analisar Algoritmos?

Analogia: projeto estrutural

Um engenheiro civil não projeta uma ponte para o dia típico de tráfego — ele projeta para o cenário mais extremo.

A análise de pior caso nos dá:

- **Garantia:** o algoritmo *nunca* será mais lento que o limite.
- **Segurança:** essencial em sistemas de tempo real.
- **Comparabilidade:** um critério justo para comparar algoritmos.



Regra de ouro da análise de pior caso

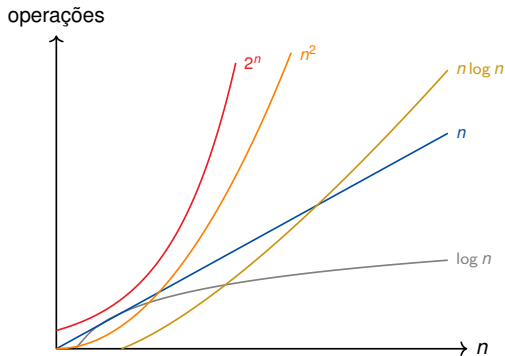
Assumimos que **laços percorrem o máximo de iterações** e que **decisões seguem o caminho de maior custo**.

A notação O captura o **limite superior assintótico**: ignoramos constantes e termos de menor ordem, focando no comportamento dominante quando $n \rightarrow \infty$.

$$3n^2 + 10n + 42 = O(n^2)$$

Também usaremos Ω (limite inferior) e Θ (limite justo) — o CLRS, capítulo 3, formaliza as três.

Quanto Custa Cada Classe de Complexidade?



Dobrar n : $\log n$ quase não muda · n dobra · n^2 quadruplica · 2^n **eleva ao quadrado o custo.**

Algoritmos Iterativos

Construção	Regra de análise
Operação básica	Atribuições, aritméticas e acessos a array custam $O(1)$.
Sequência	O custo total é o do comando mais caro (Regra da Soma).
Decisão (if/else)	No pior caso, assumimos o ramo de maior custo.
Laço (for/while)	Custo do corpo $\times$ número máximo de iterações (Regra do Produto).
Laços consecutivos	Somam-se as complexidades (Regra da Soma).
Laços aninhados	Multiplicam-se as complexidades (Regra do Produto).

Juntando tudo: operação básica + laço + decisão

ENCONTRAR-MAXIMO(A)

```
1  maximo = A[1]           // 0(1)
2  for i = 2 to A.length   // executa n - 1 vezes
3      if A[i] > maximo     // 0(1)
4          maximo = A[i]    // 0(1)
5  return maximo           // 0(1)
```

- Corpo do laço: $O(1)$. Laço executa $n - 1$ vezes: Regra do Produto $\Rightarrow O(n)$.
- Fora do laço: $O(1)$. Regra da Soma $\Rightarrow O(n) + O(1) = O(n)$.

Complexidade: $O(n)$

Exemplo 2: Selection Sort — $O(n^2)$

Padrão: o laço interno depende do índice do externo

```
SELECTION-SORT(A)
1  n = A.length
2  for i = 1 to n - 1
3      menor = i
4      for j = i + 1 to n
5          if A[j] < A[menor]
6              menor = j
7      exchange A[i] with A[menor]
```

Análise: iterações do laço interno: $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$ (progressão aritmética).

Complexidade: $O(n^2)$

Padrão: o espaço de busca é dividido por um fator constante

```
BUSCA-BINARIA(A, alvo)
1  inicio = 1
2  fim = A.length
3  while inicio <= fim
4      meio = floor((inicio + fim) / 2)
5      if A[meio] == alvo      return meio
6      elseif A[meio] < alvo   inicio = meio + 1
7      else                   fim = meio - 1
8  return NIL
```

Análise: o espaço é dividido por 2 a cada iteração: $n, n/2, n/4, \dots, 1$. Isso leva $\log_2 n$ passos.

Complexidade: $O(\log n)$

Complexidade	Padrão típico	Exemplo
$O(1)$	Acesso direto, sem laço	Acesso a $A[i]$
$O(\log n)$	Divide espaço por constante	Busca binária
$O(n)$	Um laço sobre n elementos	Busca do máximo
$O(n \log n)$	Laço $\times$ operação $\log$	n buscas binárias
$O(n^2)$	Dois laços aninhados	Selection Sort
$O(n^3)$	Três laços aninhados	Multiplicação de matrizes

Dica prática

Conte os laços aninhados que dependem de n : 1 laço $\approx O(n)$, 2 laços $\approx O(n^2)$, 3 laços $\approx O(n^3)$. Se o espaço é dividido a cada passo, pense em $\log n$.

Algoritmos Recursivos

Iterativo — já sabemos:

- Contamos laços explícitos.
- Regras da Soma e do Produto.

Recursivo — nova abordagem:

- O algoritmo chama *a si mesmo*.
- Usamos **relações de recorrência**.

Ideia central

Expressamos o tempo $T(n)$ em função do tempo para instâncias menores:

$$T(n) = \underbrace{a}_{\text{nº de chamadas}} \underbrace{T(n/b \text{ ou } n-1)}_{\text{redução}} + \underbrace{f(n)}_{\text{dividir/combinar}}$$

com um caso base de custo constante, por exemplo $T(1) = \Theta(1)$.

FATORIAL(n)

```
1  if n == 0
2      return 1
3  return n * FATORIAL(n - 1)
```

$$T(n) = T(n-1) + \Theta(1)$$

Uma chamada por nível, profundidade n .

$$O(n)$$

FIBONACCI(n)

```
1  if n <= 1
2      return n
3  return FIBONACCI(n - 1)
4      + FIBONACCI(n - 2)
```

$$T(n) = T(n-1) + T(n-2) + \Theta(1)$$

Duas chamadas por nível: a árvore dobra.

$$O(2^n)$$

Lição

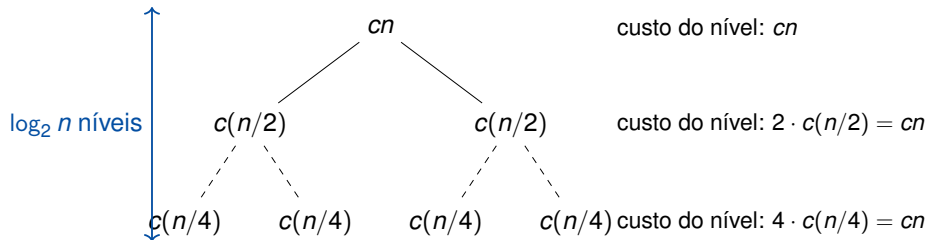
A diferença entre linear e exponencial está no **número de ramificações**.

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = floor((p + r) / 2)
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)           // custo Theta(n)
```

Recorrência: $T(n) = 2T(n/2) + \Theta(n)$

Compare com ENCONTRAR-MÁXIMO por divisão, cuja recorrência é $T(n) = 2T(n/2) + \Theta(1)$ e resulta em $O(n)$: o custo de **combinar** faz toda a diferença entre $O(n)$ e $O(n \log n)$.

Visualizando: Árvore de Recorrência do Merge Sort



Cada nível custa cn ; são $\log_2 n$ níveis. **Total:** $cn \cdot \log_2 n = \Theta(n \log n)$.

Algoritmo	Recorrência	Complexidade
Fatorial / potência ingênua	$T(n-1) + \Theta(1)$	$O(n)$
Fibonacci ingênuo	$T(n-1) + T(n-2) + \Theta(1)$	$O(2^n)$
Busca binária	$T(n/2) + \Theta(1)$	$O(\log n)$
Máximo por divisão	$2T(n/2) + \Theta(1)$	$O(n)$
Merge Sort	$2T(n/2) + \Theta(n)$	$O(n \log n)$
Quick Sort (pior caso)	$T(n-1) + \Theta(n)$	$O(n^2)$
Torre de Hanói	$2T(n-1) + \Theta(1)$	$O(2^n)$
Exponenciação rápida	$T(n/2) + \Theta(1)$	$O(\log n)$

- $T(n-1) + \Theta(1) \Rightarrow O(n)$: são n chamadas de custo constante.
- $T(n/2) + \Theta(1) \Rightarrow O(\log n)$: dividir por 2 até 1 leva $\log n$ passos.
- $2T(n-1) \Rightarrow O(2^n)$: **sinal de alerta** — ramificação sem divisão.

Teorema Mestre (Visão Rápida)

Aplica-se a recorrências da forma padrão

$$T(n) = aT(n/b) + f(n) \quad (a \geq 1, b > 1)$$

A essência: comparamos o custo de dividir/combinar $f(n)$ com o “peso das folhas” $n^{\log_b a}$. Quem domina?

Caso	Condição	Resultado
1 — Folhas dominam	$f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = \Theta(n^{\log_b a})$
2 — Empate	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$
3 — Raiz domina	$f(n) = \Omega(n^{\log_b a + \epsilon})$ + regularidade	$T(n) = \Theta(f(n))$

1. $T(n) = 2T(n/2) + n$ (Merge Sort):
folhas $n^{\log_2 2} = n$; $f(n) = n = \Theta(n)$ — empate (Caso 2). $\Theta(n \log n)$
2. $T(n) = 4T(n/2) + n$:
folhas $n^{\log_2 4} = n^2$; $f(n) = n = O(n^{2-1})$ — folhas dominam (Caso 1). $\Theta(n^2)$
3. $T(n) = T(n/2) + n$:
folhas $n^{\log_2 1} = 1$; $f(n) = n = \Omega(n^{0+1})$ — raiz domina (Caso 3). $\Theta(n)$

Voltaremos a isso

Na aula de **01/10** estudaremos o método mestre em profundidade: diagnóstico, *limitações* (quando ele não se aplica) e a árvore de recursão como método geral.

Exercícios e Encerramento

(a)

```
for i = 1 to n
  for j = 1 to n
    soma = soma + A[i][j]
```

(b)

```
i = n
while i > 1
  i = i / 2
```

(c)

```
for i = 1 to n
  j = n
  while j > 1
    j = j / 2
```

(d) Resolva a recorrência:

$$T(n) = 3T(n/2) + n$$

(Dica: $\log_2 3 \approx 1,58$.)

Como resolver

Identifique o padrão (laços? divisão? recorrência?) e aplique as regras vistas hoje. Discutiremos as respostas juntos.

Iterativos — conte os laços:

- Sem laço $\Rightarrow O(1)$
- 1 laço sobre $n \Rightarrow O(n)$
- 2 laços aninhados $\Rightarrow O(n^2)$
- 3 laços aninhados $\Rightarrow O(n^3)$
- Divide espaço pela metade
 $\Rightarrow O(\log n)$

Recursivos — monte a recorrência:

- Identifique a , n/b ou $n-1$, e $f(n)$.
- Resolva via árvore ou Teorema Mestre.

Sinais de alerta:

- $2T(n-1) \Rightarrow O(2^n)$ — exponencial!
- $T(n-1) + \Theta(n) \Rightarrow O(n^2)$

- **Quinta-feira (20/08):** Noções de corretude — invariantes de laço e terminação. Como *provar* que um algoritmo funciona?
- **Leitura recomendada:** CLRS, capítulos 2 (análise e invariantes), 3 (crescimento de funções) e 4 (recorrências).

Pergunta para levar

O Quick Sort tem pior caso $O(n^2)$, mas na prática costuma ser mais rápido que o Merge Sort ($O(n \log n)$ garantido). **Por quê?** (Pense em constantes e no caso médio.)

-  Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.
-  David Gries.
The Science of Programming.
Springer-Verlag, 1981.
-  C. A. R. Hoare.
An axiomatic basis for computer programming.
Communications of the ACM, 12(10):576–580, 1969.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.

Addison-Wesley, 1989.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.